Lecture 10 - H2, validation, errors and Collections

Saturday, October 24, 2026 10:19 PM

Lecture 10 — H2, validation, collections and Doctor association

0. Current application

Hospital API: Controller → Service → Repository.

Current change: replace ArrayList storage with H2, validate patient details and return useful error responses.

• Where are patients stored before the database conversion?

• Which layer should contain SQL queries?

• What should happen when an update is rejected?

JDBC

JDBC stands for Java Database Connectivity. It is Java's standard API for connecting to databases, executing SQL and reading results.

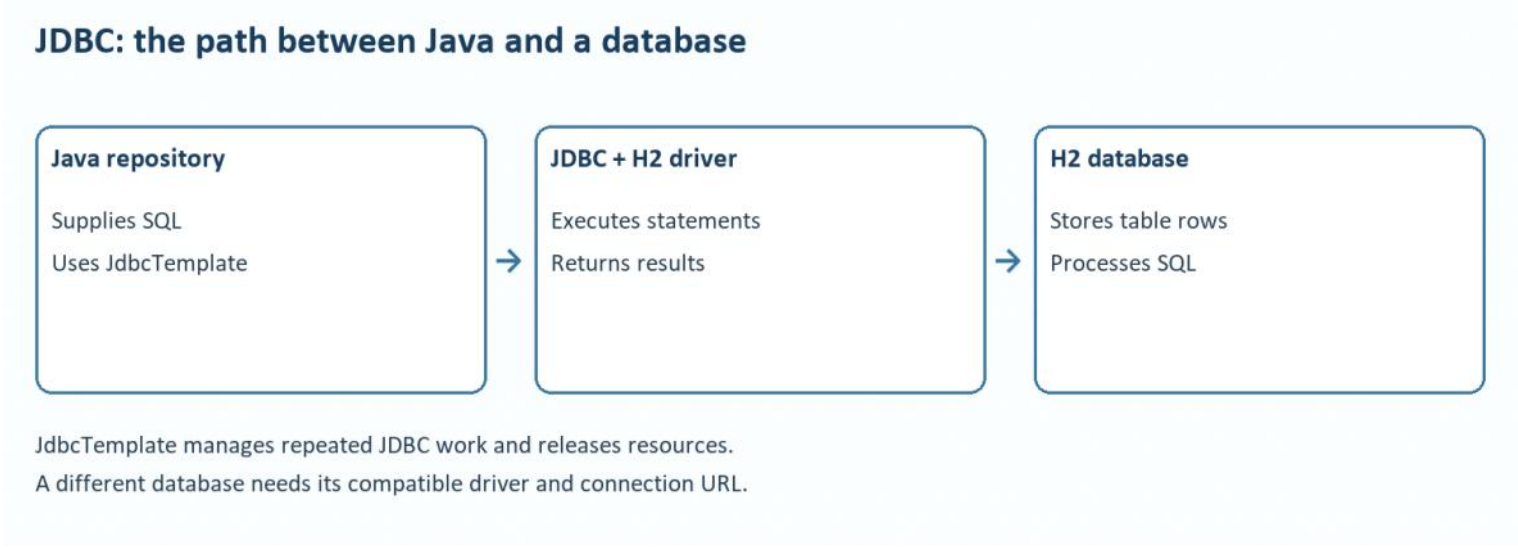
JDBC is not limited to H2. Other databases, including MySQL and PostgreSQL, use their own JDBC drivers and connection URLs. SQL syntax may vary between databases.

• JDBC: the standard Java database-access API.

• H2 driver: the database-specific implementation used to communicate with H2.

• H2: the database that executes SQL and stores rows.

• JdbcTemplate: Spring's helper that uses JDBC and manages repeated connection, statement and cleanup work.



A connection provides access to a database session. Embedded H2 can run in the application's Java process; it does not require a separate network database server.

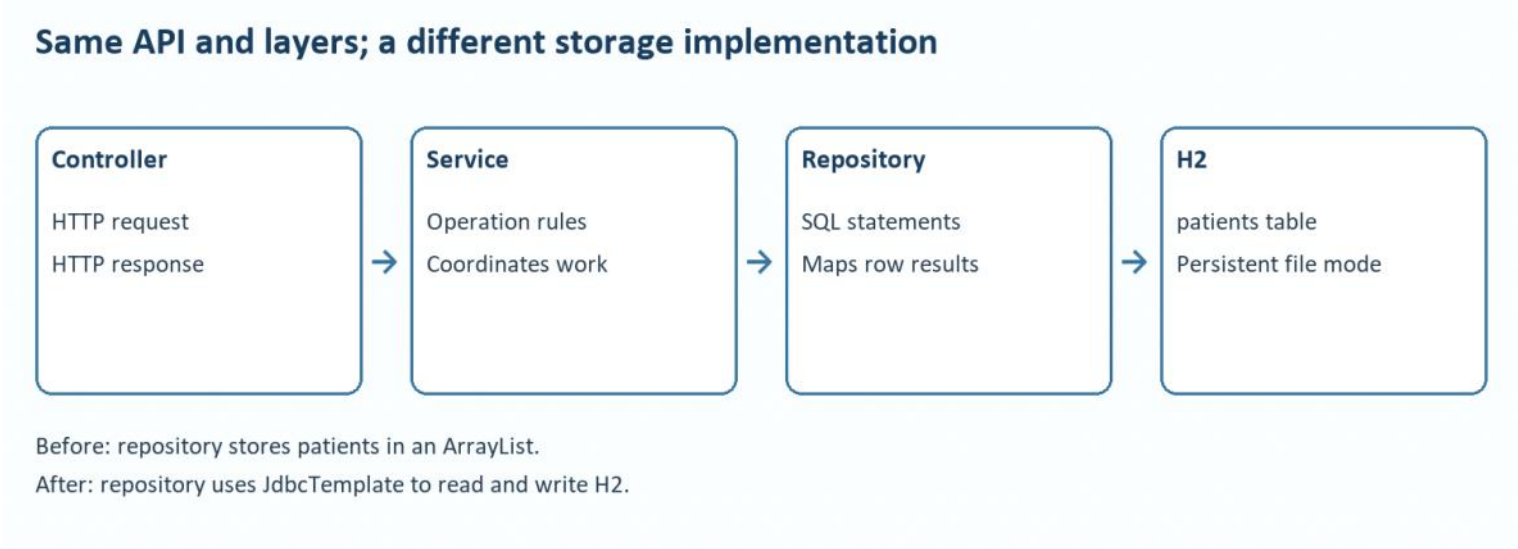
• Is JDBC itself a database?

• Why is a driver required when Java provides the JDBC API?

1. Replace ArrayList storage with H2

1A. Storage and the request path

H2 is a relational SQL database. Tables hold rows, and columns describe the stored fields. The H2 browser console is a tool for accessing the database; it is not the database itself.



The endpoint contract stays the same while the repository implementation changes. The repository executes SQL instead of updating an ArrayList.

• Does creating a Patient object automatically insert a database row?

• Which parts of the application can stay the same when storage changes?

1B. Table structure and IDs

```
CREATE TABLE IF NOT EXISTS patients (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    age INTEGER NOT NULL
);

CREATE SEQUENCE IF NOT EXISTS patient_id START WITH 1;
```

• PRIMARY KEY identifies a row uniquely.

• NOT NULL prevents a missing SQL value in that column.

• A sequence allocates IDs independently of the number of stored rows. Deleting a patient does not mean its ID should be reused as collection size + 1.

```
SELECT NEXT VALUE FOR patient_id;
```

This statement advances the sequence. Its returned value is not guaranteed to be 1 if the sequence has already been used.

1C. SQL operations

```
SELECT id, name, age FROM patients;

INSERT INTO patients (id, name, age)
VALUES (1, 'Meera', 26);

UPDATE patients
SET name = 'Meera Shah'
WHERE id = 1;

DELETE FROM patients
WHERE id = 1;
```

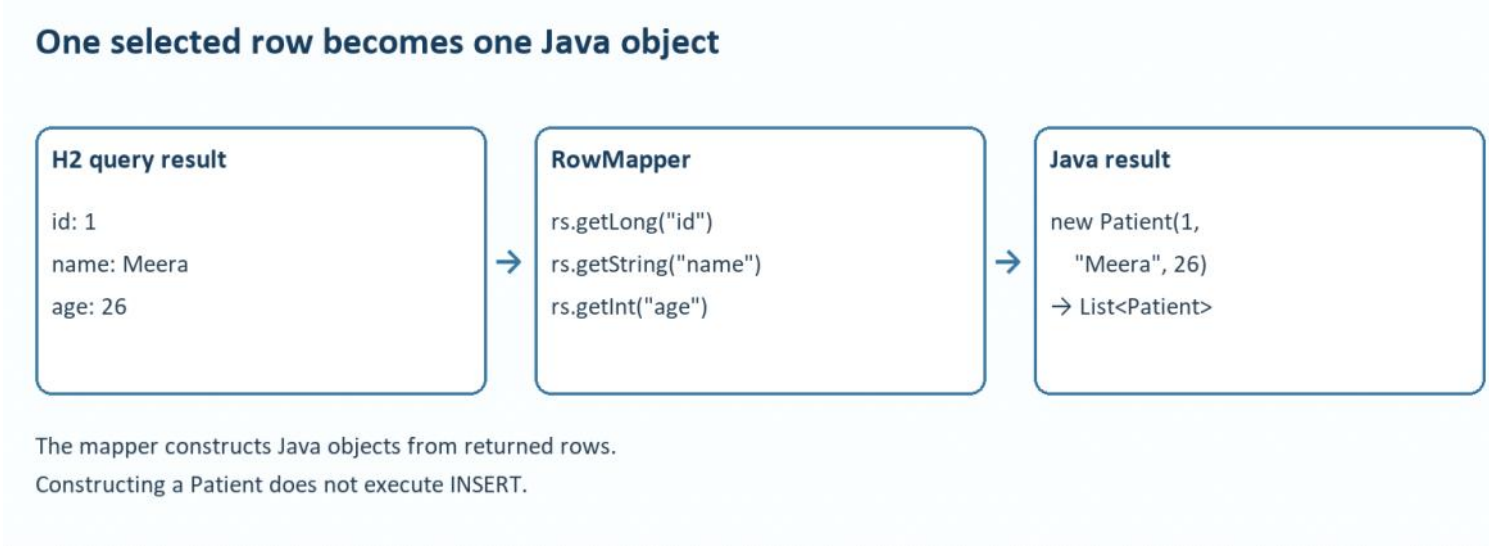
These are independent syntax examples. The Java application obtains its insert ID from the sequence; a fixed example ID must not collide with an existing row. Manually inserted IDs do not automatically advance a separate sequence.

• Which rows would UPDATE affect without WHERE?

• Which rows would DELETE affect without WHERE?

SQL practice: given a CITY table with COUNTRYCODE and POPULATION columns, select all columns for cities in the USA with population greater than 100000.

1D. From SQL rows to Java objects



A RowMapper describes how one selected database row becomes one Patient object. JdbcTemplate applies it to the returned rows.

```
private final RowMapper<Patient> patientMapper = (rs, rowNum) ->
        new Patient(
                rs.getLong("id"),
                rs.getString("name"),
                rs.getInt("age"));
```

Reading rows:

```
return jdbc.query(
        "SELECT id, name, age FROM patients ORDER BY id",
        patientMapper);
```

Allocating an ID and inserting a row:

```
long id = jdbc.queryForObject(
        "SELECT NEXT VALUE FOR patient_id", Long.class);

jdbc.update(
        "INSERT INTO patients (id, name, age) VALUES (?, ?, ?)",
        id, input.getName(), input.getAge());
```

Updating the stored name:

```
jdbc.update(
        "UPDATE patients SET name = ? WHERE id = ?",
        input.getName(), id);
```

Deleting a row:

```
int removedRows = jdbc.update(
        "DELETE FROM patients WHERE id = ?", id);
return removedRows > 0;
```

• query returns mapped rows; queryForObject reads one result value.

• JdbcTemplate.update executes INSERT, UPDATE or DELETE and returns the affected-row count.

• ? placeholders bind supplied values separately from the SQL text, in the same order as the parameters.

• What does an affected-row count of zero mean for deletion by ID?

1E. Observe the stored data

H2 console: http://localhost:8081/h2-console

```
SELECT id, name, age FROM patients ORDER BY id;
```

• Create a patient through the API and compare its returned ID with the table row.

• Update the name through the API and compare the stored row.

• Delete the patient and inspect the table again.

• With file-based H2, reopen the same database URL from the same working directory after a graceful restart to check persistence. In-memory H2 is temporary.

2. Request validation

2A. Valid patient details

• Name is required and cannot be blank.

• Age is required and must be between 0 and 120 inclusive.

• Invalid writes must leave existing data unchanged.

• Does an existing table column being NOT NULL also enforce nonblank text or an age range?

2B. Missing age versus zero

```
private Integer age;
```

Integer can represent a missing value as null. A supplied age of 0 is a number and is valid under the stated rule. Primitive int cannot represent null.

JSON conversion and business validation are different steps. Whether a JSON null is rejected or converted to a primitive default depends on deserialization settings; it is not a substitute for explicit validation rules.

2C. Annotation validation

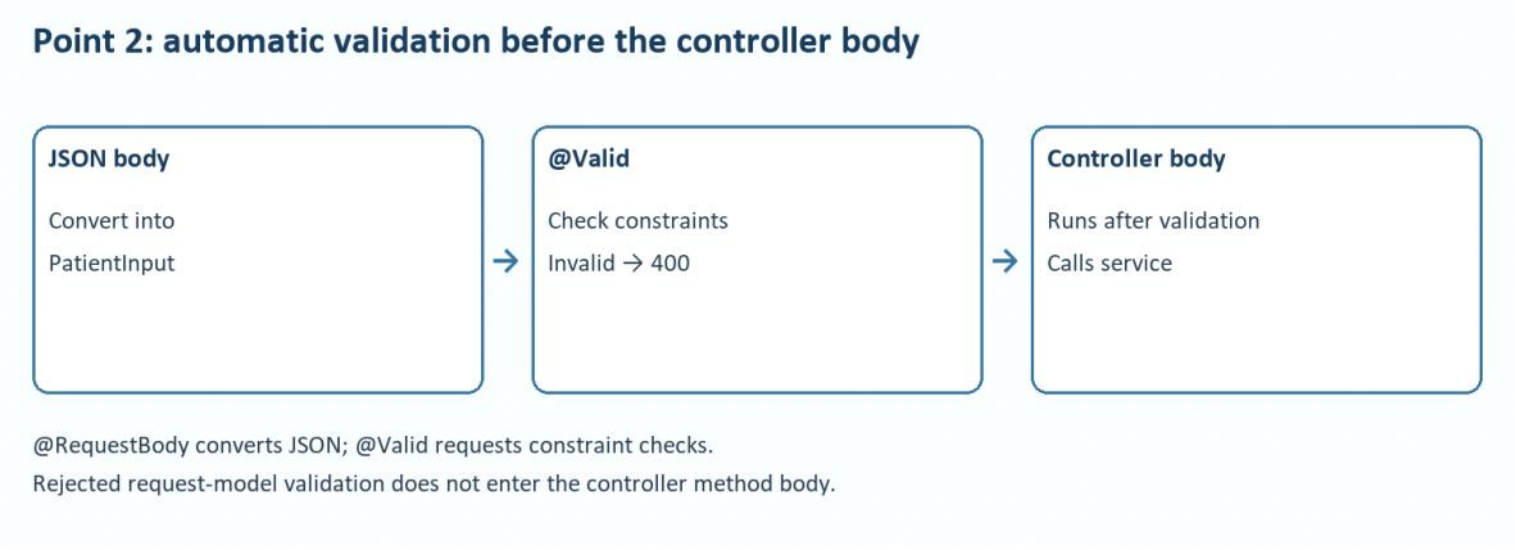
The validation starter supplies support for the request-model constraints.

```
@NotBlank(message = "Name is required")
private String name;

@NotNull(message = "Age is required")
@Min(value = 0, message = "Age must be between 0 and 120")
@Max(value = 120, message = "Age must be between 0 and 120")
private Integer age;
```

```
@Valid @RequestBody PatientInput input
```

@RequestBody converts JSON into the input model. @Valid requests constraint validation before the controller method body executes.



2D. Predict the result

```
{"name":" ","age":26}
{"name":"Meera"}
{"name":"Meera","age":-1}
{"name":"Meera","age":0}
{"name":"Meera","age":120}
{"name":"Meera","age":121}
```

• Which inputs satisfy the rules?

• Which request demonstrates the difference between missing age and zero?

• Which GET request can confirm that a rejected update changed nothing?

3. Service exceptions and controller responses

3A. Two specific failure types

InvalidPatientException: the supplied patient details violate an input rule.

PatientNotFoundException: the requested patient ID does not exist.

```
public class InvalidPatientException extends RuntimeException {
    public InvalidPatientException(String message) {
        super(message);
    }
}

public class PatientNotFoundException extends RuntimeException {
    public PatientNotFoundException(long id) {
        super("Patient " + id + " not found");
    }
}
```

super(message) stores the failure message. getMessage() reads it. Throwing an exception leaves the current execution path; subsequent statements on that path do not run.

3B. Shared service checks

validateInput checks the input object, name, name length, missing age and age range. The name length limit of 255 matches the SQL column.

```
private void validateInput(PatientInput input) {
    if (input == null) {
        throw new InvalidPatientException("Patient details are require
d");
    }
    if (input.getName() == null || input.getName().isBlank()) {
        throw new InvalidPatientException("Name is required");
    }
    if (input.getName().length() > 255) {
        throw new InvalidPatientException("Name must be at most 255 ch
aracters");
    }
    if (input.getAge() == null) {
        throw new InvalidPatientException("Age is required");
    }
    if (input.getAge() < 0 || input.getAge() > 120) {
        throw new InvalidPatientException("Age must be between 0 and 1
20");
    }
}
```

The checks run before any repository write. Null checks precede access to the input object and unboxing of age.

requirePatient checks for a matching ID and throws when no match exists.

```
private void requirePatient(long id) {
    for (Patient patient : repository.findAll()) {
        if (patient.getId() == id) {
            return;
```

```
        }
    }
    throw new PatientNotFoundException(id);
}
```

Update order:

```
validateInput(input);
requirePatient(id);
repository.updateName(id, input);
return repository.findAll();
```

• Which line is reached if validateInput throws?

• Why are both checks placed before the write?

3C. One response model

```
public class ApiError {
    private final String message;

    public ApiError(String message) {
        this.message = message;
    }

    public String getMessage() {
        return message;
    }
}
```

ApiError describes the JSON response body. It does not throw or catch an exception.

```
{"message":"Patient 99 not found"}
```

3D. Controller catches and responds

In this version, POST and PUT retain @RequestBody and omit @Valid. The service's explicit validateInput method performs validation. The model annotations remain, but are not automatically triggered by these endpoint parameters.

Annotation validation from point 2 executes before the method body. A try/catch inside that body cannot intercept its failures. This section uses manual service validation so the controller can catch the two service exceptions directly.

```
@PutMapping("/updatePatient/{id}")
public ResponseEntity<Object> updatePatient(
        @PathVariable("id") long id,
        @RequestBody PatientInput input) {
    try {
        return ResponseEntity.ok(service.updatePatient(id, input));
    } catch (InvalidPatientException exception) {
        return ResponseEntity.badRequest()
                .body(new ApiError(exception.getMessage()));
    } catch (PatientNotFoundException exception) {
        return ResponseEntity.status(404)
                .body(new ApiError(exception.getMessage()));
    }
}
```

POST /hospital/addPatient:

```
@PostMapping("/addPatient")
public ResponseEntity<Object> addPatient(@RequestBody PatientInput inp
```

```
ut) {
    try {
        return ResponseEntity.ok(service.addPatient(input));
    } catch (InvalidPatientException exception) {
        return ResponseEntity.badRequest()
                .body(new ApiError(exception.getMessage()));
    }
}
```

DELETE /hospital/deletePatients/{id}:

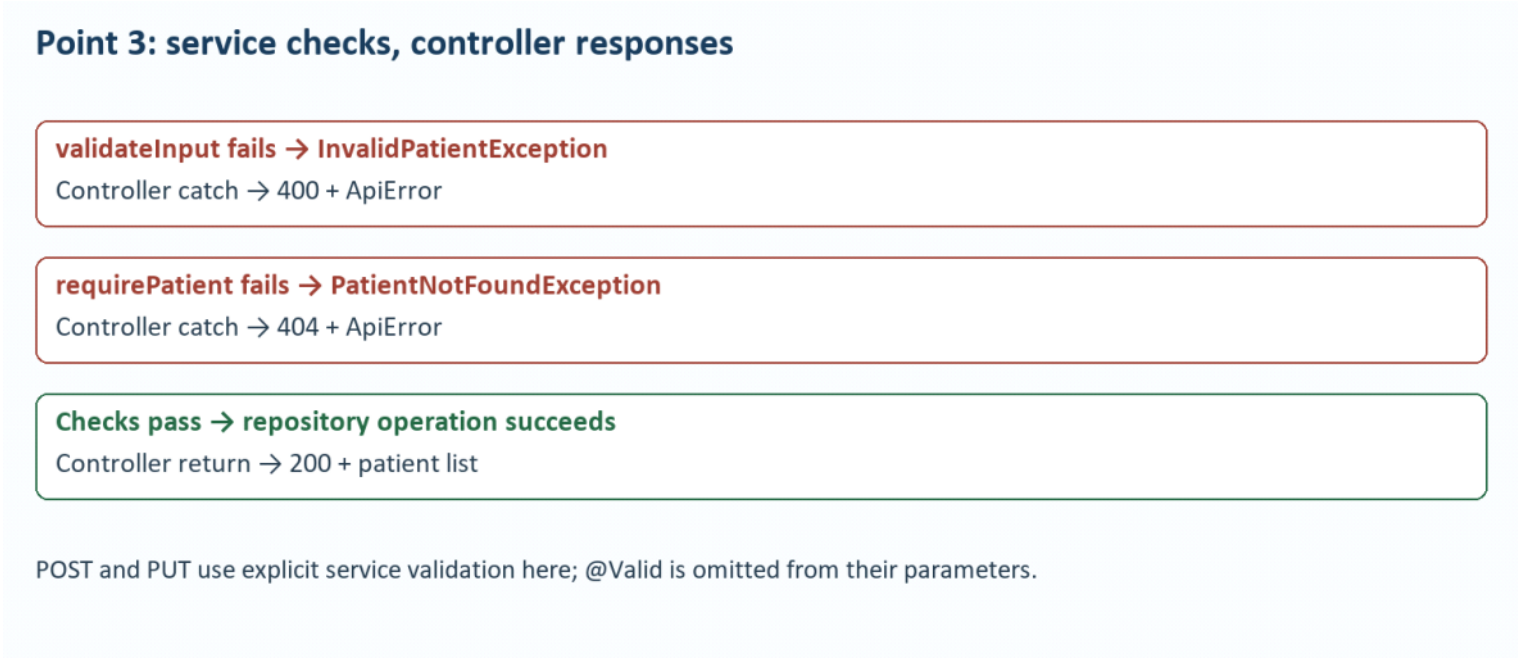
```
@DeleteMapping("/deletePatients/{id}")
public ResponseEntity<Object> deletePatient(@PathVariable("id") long i
d) {
    try {
        service.deletePatient(id);
        return ResponseEntity.ok(service.getPatients());
    } catch (PatientNotFoundException exception) {
        return ResponseEntity.status(404)
                .body(new ApiError(exception.getMessage()));
    }
}
```

• POST catches InvalidPatientException and returns 400 with ApiError.

• PUT catches invalid details as 400 and a missing patient as 404.

• DELETE calls deletion once, catches a missing patient as 404, and returns the remaining list with 200 on success.

• @ResponseStatus on an exception is not what builds this custom body. These caught failures use the status and body supplied by ResponseEntity. No shared exception handler is required for this version.



3E. API behaviour and evidence

Successful Hospital POST, name update and DELETE return 200 with the patient list. Update changes the stored name only; it retains stored age.

```
POST /hospital/addPatient
{"name":"Meera","age":26}
```

```
→ 200, patient list

PUT /hospital/updatePatient/{existingId}
{"name":"Changed","age":121}
→ 400, {"message":"Age must be between 0 and 120"}

GET /hospital/patients
→ the existing patient is unchanged

PUT /hospital/updatePatient/{missingId}
{"name":"Missing","age":26}
→ 404, {"message":"Patient ... not found"}

POST /hospital/addPatient
{"name":" ","age":-1}
→ 400, {"message":"Name is required"}

DELETE /hospital/deletePatients/{existingId}
→ 200, remaining patient list

Repeat DELETE
→ 404, missing-patient message
```

The service reports the first failed check in this simple version. Malformed JSON and invalid path-value conversion may be rejected by Spring before controller code runs.

• Does throwing an exception automatically produce an HTTP JSON response?

• Which layer detects the problem, and which layer chooses the HTTP response?

• Why can valid JSON still be invalid patient input?

• What evidence proves that a rejected write left stored data unchanged?

4. Java collections: a short recap

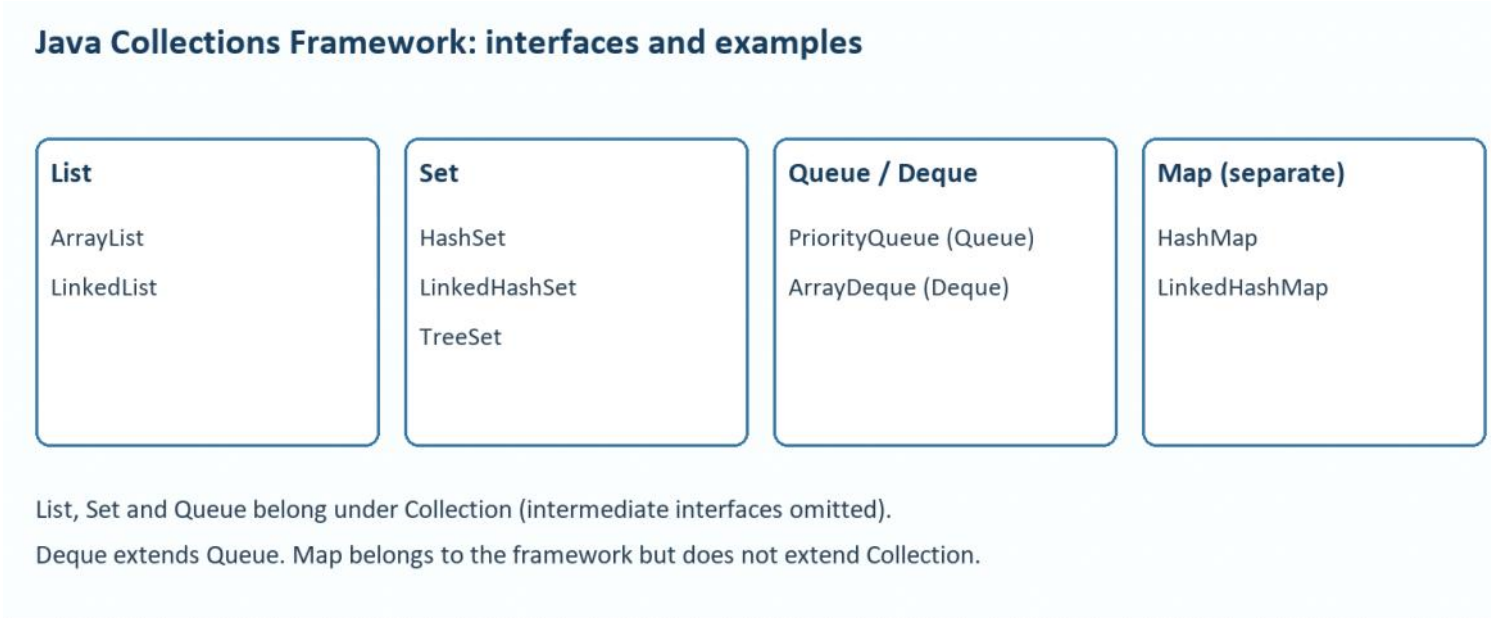
4A. Database rows and Java collections

H2 stores rows. A query returns a result set. A RowMapper converts each returned row into a Patient object, and JdbcTemplate collects the mapped results into a List<Patient>.

```
Repository → SQL query → H2 result rows
                            ↓ RowMapper
                        Patient objects
                            ↓
                        List<Patient>
```

A Patient object is one element. A List<Patient> is the collection containing those elements.

4B. Interfaces and implementations



Collection is an interface for groups of elements. List, Set and Queue are direct or indirect subinterfaces. Deque extends Queue. Map is a separate key–value interface in the Java Collections Framework; it does not extend Collection.

```
List<String> names = new ArrayList<>();
```

List describes the required behaviour; ArrayList is the chosen implementation.

Console examples use java.util classes. Each short snippet runs independently inside CollectionsDemo.main. Repeated variable declarations belong to separate runs.

```
package com.accio.collections10;
import java.util.*;

public class CollectionsDemo {
    public static void main(String[] args) {
        // One console example at a time
    }
}
```

4C. ArrayList and LinkedList

```
List<String> names =
        new ArrayList<>(List.of("Asha", "Rahul", "Asha"));

System.out.println(names.size()); // 3
System.out.println(names.get(1)); // Rahul

List<String> linked = new LinkedList<>(names);
System.out.println(linked); // [Asha, Rahul, Asha]
```

ArrayList and LinkedList: same List contract, different structure

Property / operation	ArrayList	LinkedList
Internal structure	Resizable array	Doubly linked nodes
Duplicates allowed	Yes	Yes
get(index)	O(1)	O(n)
Append at end	Amortized O(1)	O(1)
Insert at index	O(n) in general	O(n) to locate position

LinkedList insertion through an already positioned iterator can relink nodes in O(1).
Finding the insertion position by index still requires traversal.

Both implementations allow duplicates and maintain list positions. An ordered list is not necessarily sorted.

• Why can both implementations be assigned to List<String>?

• Does LinkedList insertion remain constant-time if the position must first be found by index?

4D. Queue and deque operations

```
Queue<String> waiting = new ArrayDeque<>();
waiting.offer("Asha");
waiting.offer("Rahul");

System.out.println(waiting.peek()); // Asha
System.out.println(waiting.poll()); // Asha
```

offer adds an element. peek examines the next element without removing it. poll retrieves and removes it. This ArrayDeque used as a queue follows FIFO: first in, first out.

```
Deque<String> history = new ArrayDeque<>();
history.push("Open patient");
history.push("Edit name");

System.out.println(history.pop()); // Edit name
```

A deque supports operations at both ends. push/pop use it as a stack with LIFO behaviour: last in, first out. A deque can also be used as a FIFO queue. PriorityQueue selects by priority rather than FIFO alone.

• Which collection type suits query results?

• Which suits unique assigned patient IDs?

• Which suits lookup by patient ID?

• Which suits a waiting line?

5. Wrappers, boxing/unboxing and generic classes

5A. Primitive value and wrapper reference

```
int age = 0;
Integer optionalAge = null;
```

int is a primitive type. Integer is its wrapper class. An Integer reference can be null, allowing missing age to be distinguished from a supplied value of zero.

• Why does PatientInput use Integer for a required age field?

5B. Automatic boxing and unboxing

```
int age = 26;
Integer boxed = age;  // Autoboxing: int → Integer
int unboxed = boxed; // Unboxing: Integer → int

System.out.println(boxed);   // 26
System.out.println(unboxed); // 26
```

The corresponding explicit operations are:

```
Integer boxed = Integer.valueOf(age);
int number = boxed.intValue();
```

Boxing converts a primitive to its wrapper type. Unboxing extracts a primitive value from its wrapper. Primitive widening, such as int → long, is a different conversion.

5C. Null unboxing

```
Integer missing = null; // Valid reference value
int invalid = missing;  // NullPointerException at runtime
```

The second statement needs an int value, but missing refers to no Integer object. Numeric comparisons can also trigger unboxing.

• What must be checked before comparing a possibly null Integer with zero?

5D. One class, different chosen types

Java collections already use generic types:

```
List<String>
List<Integer>
List<Patient>
```

A generic class uses one or more declared type parameters. A concrete reference type is selected when the class is used, allowing the compiler to check the values passed in and returned.

Box.java:

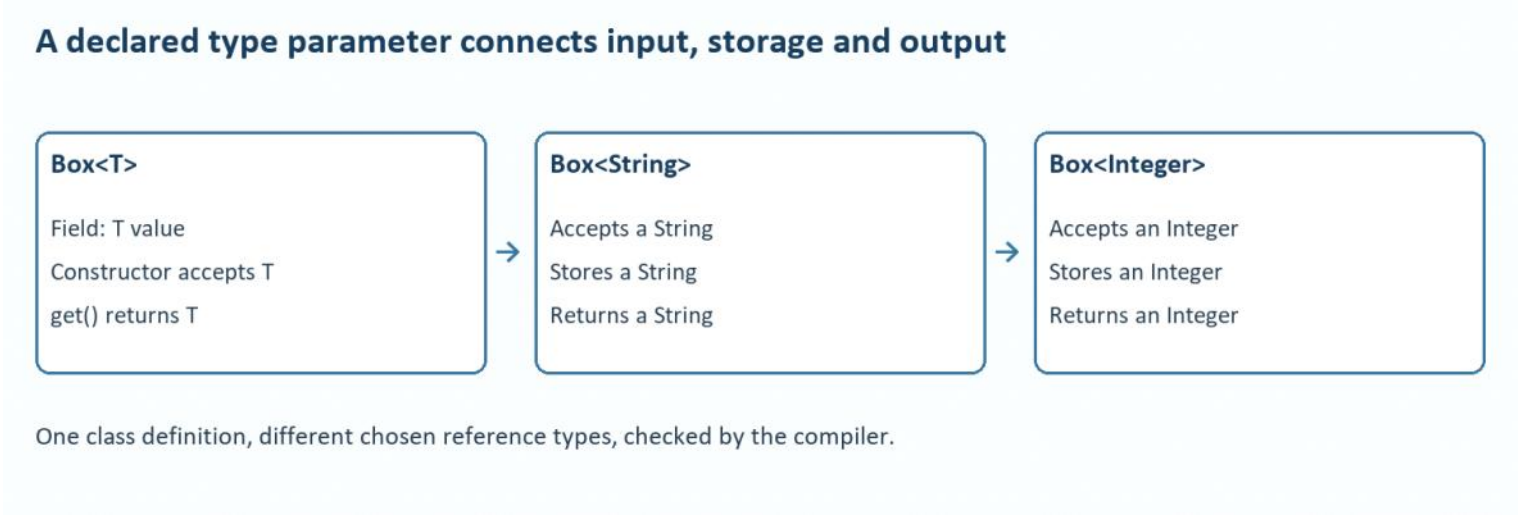
```
package com.accio.collections10;

public class Box<T> {
    private final T value;
```

```
    public Box(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}
```

T is declared by Box<T>. The same type parameter connects the stored field, constructor parameter and return type. T is a parameter name, not a Java keyword or a built-in type.



Inside CollectionsDemo.main:

```
Box<String> patientName = new Box<>("Asha");
String name = patientName.get();

Box<Integer> count = new Box<>(3);
int total = count.get();

System.out.println(name);  // Asha
System.out.println(total); // 3
```

The same class is reused with different types. Reading values needs no explicit cast.

```
// Intentionally invalid: compile-time error
// Box<Integer> wrong = new Box<>("three");
```

• Which type does get() return for Box<String>?

• Why is a String rejected when constructing Box<Integer>?

• Where must T be declared before using it in List<T>?

6. Safe removal during iteration

6A. A list change during enhanced-for traversal

```
List<String> names =
        new ArrayList<>(List.of("Asha", "Rahul", "Meera"));

for (String name : names) {
    if (name.equals("Asha")) {
```

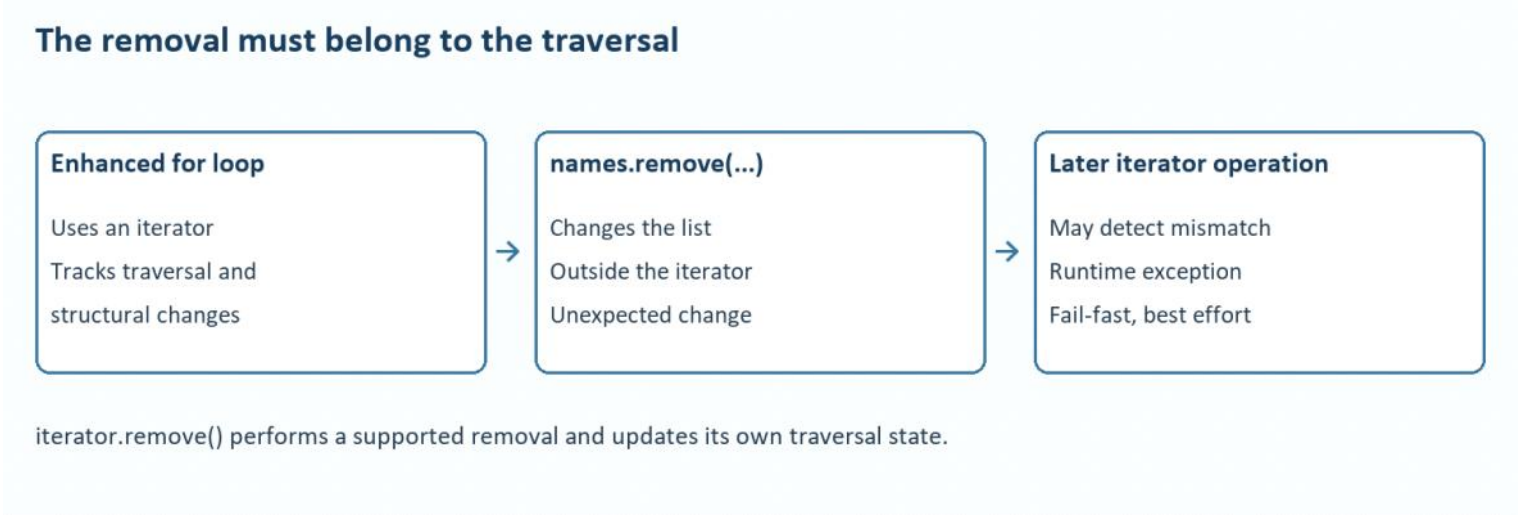
```
        names.remove(name);
    }
}
```

For this example, traversal throws ConcurrentModificationException at runtime. It is not a compilation error, and it does not require multiple threads.

An enhanced for loop over this list uses an iterator internally. Its traversal is conceptually:

```
Iterator<String> iterator = names.iterator();
while (iterator.hasNext()) {
    String name = iterator.next();
    // Loop body
}
```

6B. What the iterator detects



An ArrayList iterator tracks the list's structural modification state. Removing through names.remove changes the list outside that iterator. A later iterator operation can detect the unexpected change and throw ConcurrentModificationException.

Fail-fast detection is best effort. An unsafe loop that happens not to throw is not evidence that the pattern is safe.

• Which object is traversing the list?

• Which object performs the removal in the unsafe example?

6C. Removal through the iterator

```
List<String> names =
        new ArrayList<>(List.of("Asha", "Rahul", "Meera"));

Iterator<String> iterator = names.iterator();

while (iterator.hasNext()) {
    String name = iterator.next();
    if (name.equals("Asha")) {
        iterator.remove();
    }
}

System.out.println(names); // [Rahul, Meera]
```

hasNext checks whether another element is available. next returns it and advances traversal.

remove removes the element most recently returned by next, while keeping the iterator's traversal state consistent.

For this iterator, removal requires a preceding next and cannot be repeated twice after the same next. The operation affects the Java list; it does not execute a database DELETE.

• Why does iterator.remove take no element argument?

• What changed between the failing and working versions?

• Does changing a List<Patient> returned from a query automatically change H2 rows?

7. Map keys, Set uniqueness and object equality

7A. Unique keys and repeated values

```
Map<Long, String> byId = new LinkedHashMap<>();

byId.put(1L, "Asha");
byId.put(2L, "Asha");
byId.put(1L, "Meera");

System.out.println(byId.size());  // 2
System.out.println(byId.get(1L)); // Meera
```

Different keys can have the same value. Putting an existing key replaces its value rather than adding another entry.

```
After the first two puts:     After replacing key 1:
1 → Asha                     1 → Meera
2 → Asha                     2 → Asha
```

LinkedHashMap with its default constructor preserves insertion order. Replacing an existing key's value does not move that key to the end. HashMap does not guarantee iteration order.

• Can two patients have the same name but different IDs?

• How many entries remain after replacing the value for key 1?

7B. Equality in a Set

```
Set<String> names = new HashSet<>();
names.add("Asha");
names.add("Asha");

System.out.println(names.size()); // 1
```

HashSet does not store equal duplicates. String already defines equals and hashCode using its text.

With a custom class, equality must follow a defined rule. Initially, PatientKey has only a field and constructor:

```
package com.accio.collections10;

public final class PatientKey {
    private final long id;

    public PatientKey(long id) {
        this.id = id;
```

```
    }
}
```

```
Set<PatientKey> assigned = new HashSet<>();

System.out.println(assigned.add(new PatientKey(1))); // true
System.out.println(assigned.add(new PatientKey(1))); // true
System.out.println(assigned.size());                // 2
```

These objects contain matching IDs but are separate objects. Without overriding equals, PatientKey inherits Object's reference-based equality.

• Why does HashSet keep both objects even though both contain ID 1?

7C. Define logical identity

For this example, two PatientKey objects represent the same logical key when their IDs match.

PatientKey.java, with equality and hashing defined:

```
package com.accio.collections10;

import java.util.Objects;

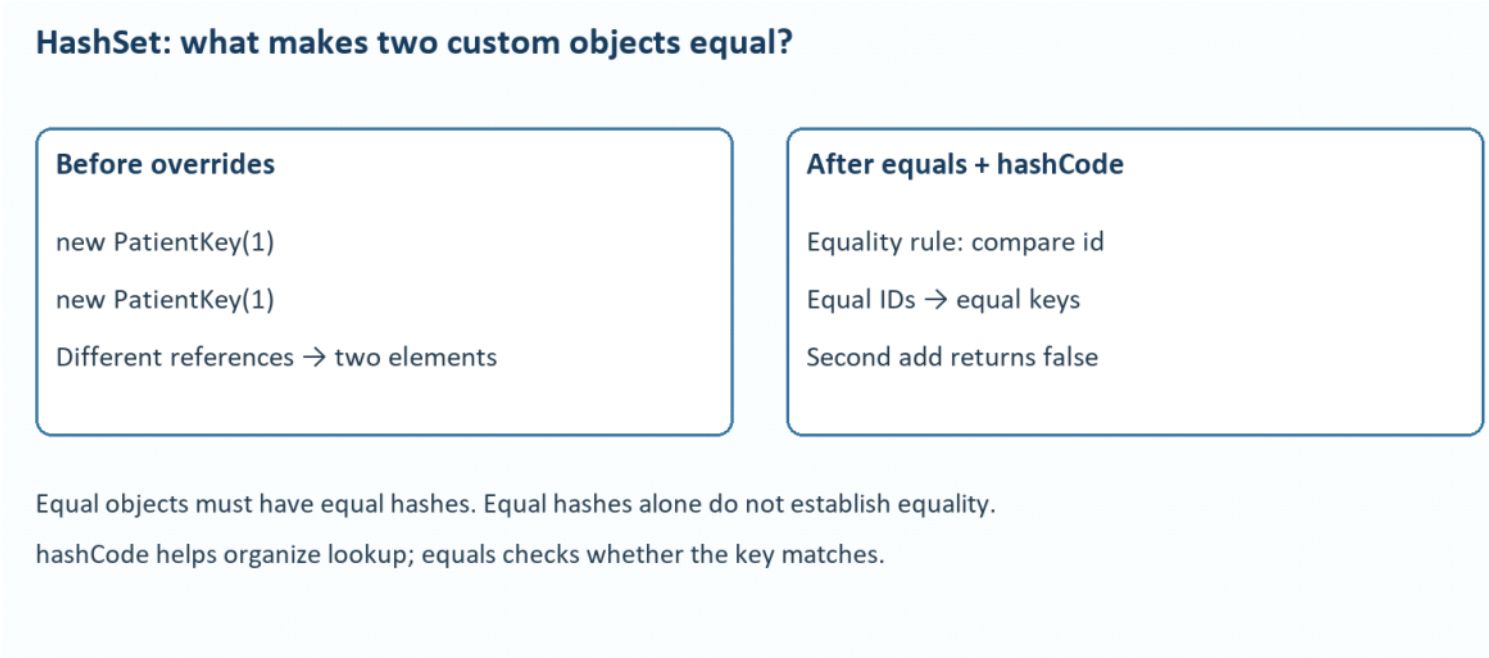
public final class PatientKey {
    private final long id;

    public PatientKey(long id) {
        this.id = id;
    }

    @Override
    public boolean equals(Object other) {
        if (this == other) return true;
        if (!(other instanceof PatientKey)) return false;
        PatientKey that = (PatientKey) other;
        return id == that.id;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

equals checks logical equality. A hash-based collection uses hashCode to organize lookup and equals to distinguish matching objects.



• Equal objects must have the same hash code.

• The same hash code does not guarantee equality: different objects can have a hash collision.

• Fields used for equality and hashing should not change while an object is a hash key or HashSet element. This example uses a final ID.

With the completed class, a fresh Set behaves as follows:

```
Set<PatientKey> assigned = new HashSet<>();

System.out.println(assigned.add(new PatientKey(1))); // true
System.out.println(assigned.add(new PatientKey(1))); // false
System.out.println(assigned.add(new PatientKey(2))); // true
System.out.println(assigned.size());                // 2
```

• Why does creating a new object no longer guarantee a new Set element?

• Why is patient ID a better identity choice than patient name in this application?

8. Collections utilities and a patient-name report

8A. Collection and Collections

Collection is an interface for groups of elements. Collections is a utility class containing static methods that operate on collections.

```
List<String> names =
        new ArrayList<>(List.of("Rahul", "Asha", "Asha"));

Collections.sort(names);
System.out.println(names); // [Asha, Asha, Rahul]

int count = Collections.frequency(names, "Asha");
System.out.println(count); // 2

Collections.reverse(names);
System.out.println(names); // [Rahul, Asha, Asha]
```

• sort and reverse operate on a List. The example uses a mutable ArrayList because these operations change element order.

• frequency accepts a Collection and counts elements equal to one supplied value. A Map itself is not a Collection, but map.values() is.

• Reversing existing order is different from sorting in descending order.

8B. Exercise — counts for every patient name

Implement GET /hospital/nameCounts. Return a JSON object containing each exact patient name and the number of patients with that name.

```
Patient names: John, Jane, Jane

Expected response:
{"John":1,"Jane":2}
```

Requirements:

• Read patients from the existing H2-backed repository.

• Duplicate names are allowed. John and john count separately.

• An empty patient collection returns {}.

• Repeated calls must not change stored patient data.

• Which collection associates each name with a count?

• What is the count when a name appears for the first time?

8C. Service calculation and endpoint

HospitalPatientService imports:

```
import java.util.Map;
import java.util.LinkedHashMap;
```

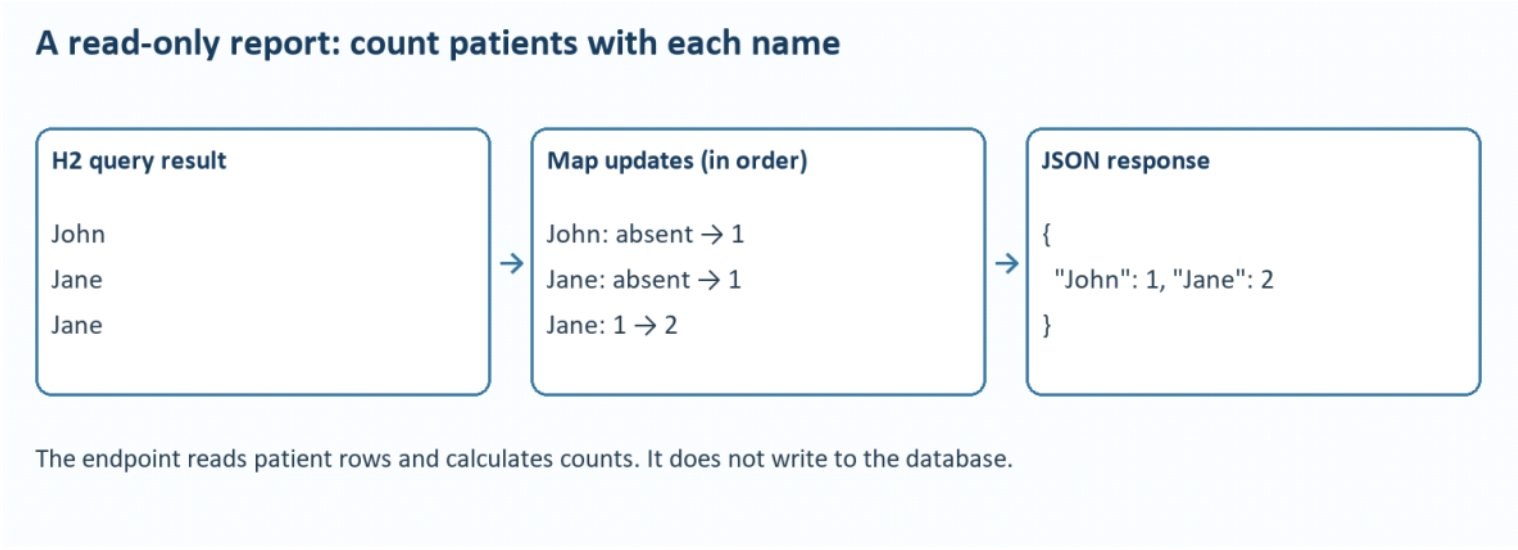
```
public Map<String, Integer> countPatientsByName() {
    Map<String, Integer> counts = new LinkedHashMap<>();

    for (Patient patient : repository.findAll()) {
        String name = patient.getName();
        Integer previous = counts.get(name);
        counts.put(name, previous == null ? 1 : previous + 1);
    }

    return counts;
}
```

HospitalController uses java.util.Map:

```
@GetMapping("/nameCounts")
public Map<String, Integer> nameCounts() {
    return service.countPatientsByName();
}
```



The map exists for this calculation. It is not the patient database. The endpoint derives a response from stored rows without writing to them.

• Which request can confirm the patient records remain unchanged?

9. Doctor–Patient association in H2

Problem statement

Extend the Hospital API so existing patients can be assigned to doctors. Store both doctor records and assignments in the same H2 database as the patients.

• Doctor fields: id and name. Create the doctors table at startup and insert two sample doctors through the H2 Console: 1 / Dr Asha and 2 / Dr Meera.

• A doctor can have many patients. A patient is unassigned or assigned to one doctor at a time.

• PUT /doctors/{doctorId}/patients/{patientId} assigns an existing patient and returns 200 with the doctor and their current patients.

• GET /doctors/{doctorId}/patients returns 200 with that doctor's patient list. An existing doctor with no assigned patients returns [].

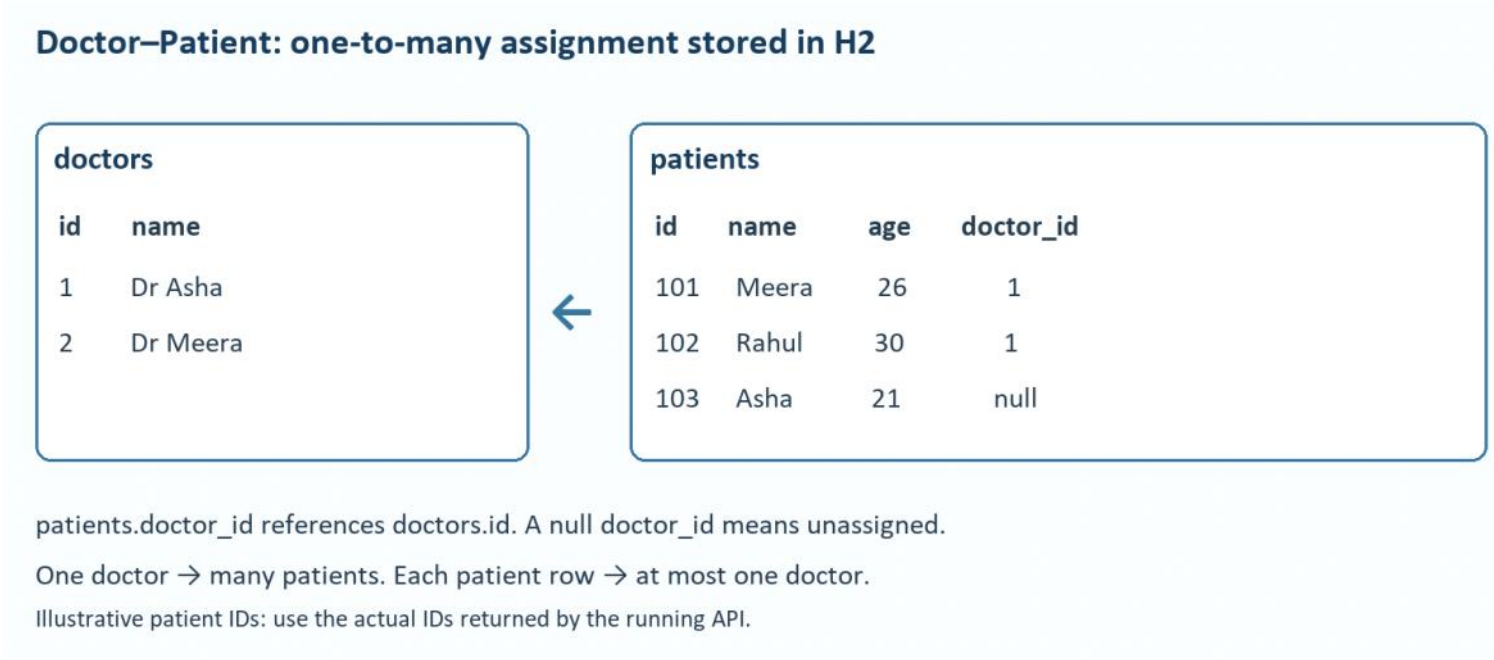
• A missing doctor or patient returns 404 with a useful message. A failed assignment must leave existing assignments unchanged.

• Repeating assignment to the same doctor must not duplicate the patient. Assigning a different doctor moves the patient to that doctor.

• Assignments must survive restart. Later reads must show current patient names and exclude deleted patients.

Use the existing JdbcTemplate, constructor injection, service exceptions and controller-level try/catch approach.

9A. The persistent relationship



patients.doctor_id records the assignment. A null value means unassigned. Many patient rows may contain the same doctor ID, but one patient row contains at most one assigned doctor ID.

The foreign key ensures that a non-null doctor_id refers to an existing row in doctors. It does not require doctor_id itself to be unique.

• Which column records the relationship?

• Why can a doctor have several patients without storing several IDs in one patient row?

9B. Table creation and sample doctors

Append to schema.sql after the existing patients table and sequence:

```
CREATE TABLE IF NOT EXISTS doctors (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255) NOT NULL
);

ALTER TABLE patients
    ADD COLUMN IF NOT EXISTS doctor_id BIGINT;

ALTER TABLE patients
    ADD CONSTRAINT IF NOT EXISTS fk_patients_doctor
    FOREIGN KEY (doctor_id) REFERENCES doctors(id);
```

ALTER TABLE extends the existing patient table. Existing patient IDs, names and ages remain; doctor_id initially has a null value. IF NOT EXISTS permits this schema setup to run on later startups.

H2 Console: http://localhost:8081/h2-console

The JDBC URL must match the application's existing file database URL. The configured username is sa with a blank password.

Inspect existing doctor records:

```
SELECT id, name FROM doctors ORDER BY id;
```

For a database where doctor IDs 1 and 2 are absent, insert the sample rows once:

```
INSERT INTO doctors (id, name)
VALUES (1, 'Dr Asha'), (2, 'Dr Meera');
```

These rows persist in the file database. Repeating the same INSERT with existing IDs fails because the primary keys already exist. This version uses manual sample-data insertion, not an automatic doctor seed script.

9C. Doctor model and missing-doctor exception

Doctor's patient list is a response collection loaded from H2. Assignments are persisted by SQL, not by adding an object to that Java list.

Model/Doctor.java:

```
package com.accio.springProject.Hospital.Model;

import com.accio.springProject.Hospital.Entity.Patient;
import java.util.ArrayList;
import java.util.List;

public class Doctor {
    private final long id;
    private final String name;
    private List<Patient> patients = new ArrayList<>();

    public Doctor(long id, String name) {
        this.id = id;
        this.name = name;
    }

    public long getId() { return id; }
    public String getName() { return name; }
    public List<Patient> getPatients() { return patients; }
    public void setPatients(List<Patient> patients) {
        this.patients = patients;
    }
}
```

Exception/DoctorNotFoundException.java:

```
package com.accio.springProject.Hospital.Exception;

public class DoctorNotFoundException extends RuntimeException {
    public DoctorNotFoundException(long id) {
        super("Doctor " + id + " not found");
    }
}
```

PatientNotFoundException and ApiError are reused from the earlier Hospital endpoints.

9D. Repository and named row mappers

The doctorMapper field maps a doctor row; patientMapper maps each selected patient row.

Repository/DoctorRepository.java:

```
package com.accio.springProject.Hospital.Repository;

import com.accio.springProject.Hospital.Model.Doctor;
import com.accio.springProject.Hospital.Entity.Patient;
import java.util.List;
```

```
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Repository;

@Repository
public class DoctorRepository {
    private final JdbcTemplate jdbc;

    private final RowMapper<Doctor> doctorMapper = (rs, rowNum) ->
            new Doctor(rs.getLong("id"), rs.getString("name"));

    private final RowMapper<Patient> patientMapper = (rs, rowNum) ->
            new Patient(rs.getLong("id"), rs.getString("name"), rs.get
Int("age"));

    public DoctorRepository(JdbcTemplate jdbc) {
        this.jdbc = jdbc;
    }

    public Doctor findById(long id) {
        List<Doctor> matches = jdbc.query(
                "SELECT id, name FROM doctors WHERE id = ?",
                doctorMapper,
                id);
        return matches.isEmpty() ? null : matches.get(0);
    }

    public boolean assignPatient(long doctorId, long patientId) {
        int changedRows = jdbc.update(
                "UPDATE patients SET doctor_id = ? WHERE id = ?",
                doctorId, patientId);
        return changedRows > 0;
    }

    public List<Patient> findPatientsByDoctorId(long doctorId) {
        return jdbc.query(
                "SELECT id, name, age FROM patients WHERE doctor_id =
? ORDER BY id",
                patientMapper,
                doctorId);
    }
}
```

• findById returns the mapped doctor or null when no row matches.

• assignPatient updates doctor_id on an existing patient row. Zero affected rows indicates that the patient was absent.

• findPatientsByDoctorId queries current patient records and maps them to List<Patient>.

• Why does repeating the same assignment avoid duplicate patients?

9E. Service rules

Service/DoctorService.java:

```
package com.accio.springProject.Hospital.Service;

import com.accio.springProject.Hospital.Model.Doctor;
import com.accio.springProject.Hospital.Entity.Patient;
import com.accio.springProject.Hospital.Repository.DoctorRepository;
import com.accio.springProject.Hospital.Exception.DoctorNotFoundExcept
ion;
import com.accio.springProject.Hospital.Exception.PatientNotFoundExcep
tion;
import java.util.List;
import org.springframework.stereotype.Service;

@Service
public class DoctorService {
    private final DoctorRepository repository;

    public DoctorService(DoctorRepository repository) {
```

```
        this.repository = repository;
    }

    public Doctor assign(long doctorId, long patientId) {
        Doctor doctor = requireDoctor(doctorId);
        if (!repository.assignPatient(doctorId, patientId)) {
            throw new PatientNotFoundException(patientId);
        }
        doctor.setPatients(repository.findPatientsByDoctorId(doctorId)
);
        return doctor;
    }

    public List<Patient> getPatients(long doctorId) {
        requireDoctor(doctorId);
        return repository.findPatientsByDoctorId(doctorId);
    }

    private Doctor requireDoctor(long id) {
        Doctor doctor = repository.findById(id);
        if (doctor == null) {
            throw new DoctorNotFoundException(id);
        }
        return doctor;
    }
}
```

```
Assignment:
Check doctor → update patient row → read assigned patients
                                        ↓
                          Doctor response with List<Patient>

Read assigned patients:
Check doctor → select current patient rows → return list
```

Checking the doctor first prevents assigning a nonexistent doctor. When the patient update affects no rows, the service throws PatientNotFoundException. A Doctor object is assembled for the response; it is not stored in a repository Map.

9F. HTTP endpoints

Controller/DoctorController.java:

```
package com.accio.springProject.Hospital.Controller;

import com.accio.springProject.Hospital.Model.ApiError;
import com.accio.springProject.Hospital.Service.DoctorService;
import com.accio.springProject.Hospital.Exception.DoctorNotFoundExcept
ion;
import com.accio.springProject.Hospital.Exception.PatientNotFoundExcep
tion;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/doctors")
public class DoctorController {
    private final DoctorService service;

    public DoctorController(DoctorService service) {
        this.service = service;
    }

    @PutMapping("/{doctorId}/patients/{patientId}")
    public ResponseEntity<Object> assign(
            @PathVariable("doctorId") long doctorId,
            @PathVariable("patientId") long patientId) {
        try {
            return ResponseEntity.ok(service.assign(doctorId, patientI
d));
        } catch (DoctorNotFoundException exception) {
            return ResponseEntity.status(404)
                    .body(new ApiError(exception.getMessage()));
        } catch (PatientNotFoundException exception) {
            return ResponseEntity.status(404)
                    .body(new ApiError(exception.getMessage()));
        }
```

```
    }

    @GetMapping("/{doctorId}/patients")
    public ResponseEntity<Object> getPatients(@PathVariable("doctorId"
) long doctorId) {
        try {
            return ResponseEntity.ok(service.getPatients(doctorId));
        } catch (DoctorNotFoundException exception) {
            return ResponseEntity.status(404)
                    .body(new ApiError(exception.getMessage()));
        }
    }
}
```

The controller catches the specific service exceptions and returns 404 with ApiError. Successful assignment returns the doctor and patients; reading assignments returns only the patient list.

9G. Verify the behaviour

Use the actual patient ID returned by the existing create endpoint.

```
GET /doctors/1/patients
→ 200 [] when none are assigned

POST /hospital/addPatient
{"name":"Meera","age":26}
→ record the new patient's ID

PUT /doctors/1/patients/{patientId}
→ 200, doctor with current patients

Repeat the same PUT
→ patient appears once

PUT /hospital/updatePatient/{patientId}
{"name":"Meera Shah","age":26}
GET /doctors/1/patients
→ current name Meera Shah

Restart with the same database URL and working directory.
GET /doctors/1/patients
→ assignment remains

PUT /doctors/{missingDoctorId}/patients/{patientId}
→ 404; the previous assignment remains

PUT /doctors/1/patients/{missingPatientId}
→ 404

PUT /doctors/2/patients/{patientId}
→ patient moves to doctor 2

DELETE /hospital/deletePatients/{patientId}
GET /doctors/2/patients
→ deleted patient is absent
```

Inspect the stored data:

```
SELECT id, name FROM doctors ORDER BY id;
SELECT id, name, age, doctor_id FROM patients ORDER BY id;
```

- Does changing Doctor.patients alone update the database?

- Why does a renamed patient show the new name in the assigned list?

- How does a missing doctor differ from an existing doctor with no patients?

- Which data survives restart, and what evidence confirms that it did?